Sub-Project Software Design Document

for

User Identity, Profile and Portfolio Management

Module 1 — National Freelance & Skill Verification Platform

Version 1.0 approved

Prepared by Abdulah Tariq

National University of Computer and Emerging Sciences

27th March, 2026

Contents

1. Introduction

1.1 Purpose

This Sub-Project Software Design Document (SP-SDS) describes the architecture, system design, data structures, component design, and human interface design for Module 1 — the User Identity, Profile and Portfolio Management Module of the National Freelance and Skill Verification Platform. This document translates the requirements defined in the SP-SRS into a detailed blueprint for implementation, serving as the primary reference for developers writing code for this module.

1.2 Scope

The User Identity, Profile and Portfolio Management Module (Module 1) is a foundational sub-system of the National Freelance and Skill Verification Platform. It provides user registration and authentication, professional profile management, portfolio and project sample management, work history tracking, and a review and rating system.

This module acts as the central identity and profile data provider for the entire platform. It exposes REST APIs consumed by dependent modules including the AI Matching Module (Module 4), the Communication Module (Module 6), the Payment Module (Module 7), the Admin Module (Module 8), the Analytics Module (Module 9), and the Gamification Module (Module 11).

1.3 Overview

This document is organized as follows:

- Section 1 — Introduction: Purpose, scope, references, and definitions.
- Section 2 — System Overview: General description of the module and its context.
- Section 3 — System Architecture: Architectural design, decomposition, and design rationale.
- Section 4 — Data Design: Data structures, entity relationships, and data dictionary.
- Section 5 — Component Design: Detailed pseudocode and logic for all major components.
- Section 6 — Human Interface Design: UI overview, screen wireframes, and screen object actions.
- Section 7 — Requirements Matrix: Traceability from design components to SP-SRS requirements.
- Section 8 — Appendices: Supporting information.

1.4 Reference Material

- Sub-Project Software Requirements Specification (SP-SRS) — Module 1, Version 1.0, 27th March 2026
- Sub-Project Management Plan (SPMP) — Module 1, Version 1.0, 27th March 2026
- Sub-Project Requirements Management Plan (SP-RMP) — Module 1, Version 1.0, 27th March 2026
- Kickoff Meeting Minutes — 26th March 2026
- IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions
- Module Decomposition Document — National Freelance and Skill Verification Platform (11 Modules)

1.5 Definitions and Acronyms

Term / Acronym	Definition
SP-SDS	Sub-Project Software Design Document
SP-SRS	Sub-Project Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer — architectural style for web APIs
JWT	JSON Web Token — compact token used for stateless authentication
RBAC	Role-Based Access Control
HTTPS	Hypertext Transfer Protocol Secure
UI	User Interface
FR-PM	Functional Requirement — Profile Management (Module 1 identifier)

2. System Overview

The National Freelance and Skill Verification Platform is a web-based system designed to establish trust, transparency, and secure collaboration between freelancers and clients in Pakistan's digital economy. The platform consists of 11 distinct modules, each responsible for a specific domain of functionality. Module 1 — the User Identity, Profile and Portfolio Management Module — serves as the identity foundation of the entire platform.

Module 1 is responsible for managing who users are on the platform. It handles the complete lifecycle of a user's digital identity: from initial registration and authentication, through profile creation and management, to the display of portfolios, work history, and reputation indicators. Every other module that needs to know about a user — their name, role, skills, availability, or trustworthiness — depends on data provided by this module.

The module is built using a three-tier web architecture consisting of a React.js frontend, a Node.js/Express backend, and a shared centralized PostgreSQL database used across all platform modules. Each module owns and manages its own tables within this shared schema. The backend exposes RESTful APIs that serve both the frontend and other platform modules. Authentication is handled using JSON Web Tokens (JWT), and role-based access control is enforced at the API layer to ensure that freelancers, clients, and administrators can only access features appropriate to their role.

3. System Architecture

3.1 Architectural Design

Module 1 follows a layered three-tier architecture pattern, which separates the system into three distinct layers: the Presentation Layer, the Application Layer, and the Data Layer.

Layer	Technology	Responsibility
Presentation Layer	React.js (SPA)	Renders UI, handles user interactions, communicates with backend via HTTP
Application Layer	Node.js + Express.js	Business logic, authentication, RBAC enforcement, REST API endpoints
Data Layer	Shared centralized PostgreSQL database (shared across all platform modules). Module 1 owns the Users, Profiles, Portfolio Items, Work History, and Reviews tables.	Persistent storage of users, profiles, portfolios, work history, reviews

- The React.js frontend communicates with the Node.js backend exclusively via HTTPS REST API calls.
- The backend processes requests, applies business rules and RBAC, then interacts with the shared centralized PostgreSQL database via Module 1's own tables.
- External modules communicate with Module 1 through the REST API layer using agreed-upon endpoints. Direct cross-module database queries are not permitted; all inter-module data access is performed through APIs to maintain loose coupling.
- JWT tokens are issued on login and validated on every protected API request.

3.2 Decomposition Description

The module is decomposed into six subsystems:

3.2.1 Authentication Subsystem

Handles all user registration and login operations. Responsible for validating credentials, hashing passwords, issuing JWT tokens, and enforcing role-based access on all protected routes.

- Components: RegisterController, LoginController, AuthMiddleware, TokenService
- Inputs: User credentials (email, password, role) | Outputs: JWT access token, user role, user ID

3.2.2 Profile Management Subsystem

Manages the creation, retrieval, updating, and deletion of user profiles. Stores personal information, skill tags, profile pictures, and aggregated badge data from other modules.

- Components: ProfileController, ProfileService, ProfileRepository
- Inputs: Profile form data; badge and trust score data written to the shared database by Modules 2, 10, and 11 | Outputs: Profile data exposed via REST API to Modules 3, 4, 6, 7, 8, 9, 11

3.2.3 Portfolio Management Subsystem

Enables freelancers to upload, manage, and publicly display portfolio items. Handles file storage, metadata management, and access control.

- Components: PortfolioController, PortfolioService, FileUploadHandler, PortfolioRepository

3.2.4 Work History Subsystem

Maintains a structured record of professional experience and completed projects for freelancers. Supports full CRUD operations on work history entries.

- Components: WorkHistoryController, WorkHistoryService, WorkHistoryRepository

3.2.5 Reviews and Ratings Subsystem

Allows clients to submit ratings and written reviews for freelancers. Calculates and maintains the running average rating displayed on profiles.

- Components: ReviewController, ReviewService, RatingCalculator, ReviewRepository

3.2.6 API Gateway Layer

All incoming requests pass through this layer which enforces JWT validation, applies RBAC middleware, and routes requests to the appropriate subsystem controller.

- Components: Router, AuthMiddleware, RBACMiddleware, ErrorHandler

3.3 Design Rationale

- **Separation of Concerns:** Each layer has a clearly defined responsibility, making the codebase easier to maintain and test independently.
- **Scalability:** The backend API layer can be scaled independently from the frontend, and the database layer can be optimized without impacting business logic.
- **Interoperability:** A REST API-based backend allows other modules to consume Module 1 data without tight coupling.
- **Industry Alignment:** React.js + Node.js + PostgreSQL is a widely adopted stack aligning with academic and industry best practices.

Alternatives considered: Monolithic MVC (rejected — poor separation for inter-module integration) and Microservices (rejected — overly complex for an academic project with a small team). A shared centralized database was adopted over separate per-module databases following agreement with the Integration Group, as it simplifies deployment, reduces infrastructure overhead, and is appropriate for the academic project scale.

4. Data Design

4.1 Data Description

The module manages five primary data entities: Users, Profiles, Portfolio Items, Work History Entries, and Reviews. These are stored within Module 1's dedicated tables in the shared centralized PostgreSQL database used across all platform modules.

- A User has exactly one Profile (one-to-one).
- A User (Freelancer) may have zero or more Portfolio Items (one-to-many).
- A User (Freelancer) may have zero or more Work History Entries (one-to-many).
- A User (Freelancer) may have zero or more Reviews submitted by Clients (one-to-many).
- External badge and trust score data from other modules is stored as JSONB fields within the Profile entity.

4.2 Data Dictionary

4.2.1 Users Table

Field Name	Data Type	Constraints	Description
user_id	UUID	PK, NOT NULL	Unique identifier for each user (auto-generated)
email	VARCHAR(255)	UNIQUE, NOT NULL	User's email address used for login
password_hash	VARCHAR(255)	NOT NULL	bcrypt-hashed password
role	ENUM	NOT NULL	User role: 'freelancer', 'client', or 'admin'
is_active	BOOLEAN	DEFAULT true	Indicates whether the account is active
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp
updated_at	TIMESTAMP	DEFAULT NOW()	Last update timestamp

4.2.2 Profiles Table

Field Name	Data Type	Constraints	Description
profile_id	UUID	PK, NOT NULL	Unique identifier for the profile
user_id	UUID	FK (Users), UNIQUE	References the owning user
full_name	VARCHAR(150)	NOT NULL	User's full display name
bio	TEXT	NULLABLE	Short professional biography
location	VARCHAR(100)	NULLABLE	User's city or country

Field Name	Data Type	Constraints	Description
profile_picture_url	VARCHAR(500)	NULLABLE	URL path to the stored profile image
skills	TEXT[]	NULLABLE	Array of skill tags entered by the user
badges	JSONB	NULLABLE	Badges received from Modules 2, 10, 11
trust_score	DECIMAL(4,2)	NULLABLE	Trust score received from Module 11
avg_rating	DECIMAL(3,2)	DEFAULT 0.00	Computed average rating from reviews
updated_at	TIMESTAMP	DEFAULT NOW()	Last profile update timestamp

4.2.3 Portfolio Items Table

Field Name	Data Type	Constraints	Description
item_id	UUID	PK, NOT NULL	Unique identifier for the portfolio item
user_id	UUID	FK (Users), NOT NULL	References the freelancer who owns this item
title	VARCHAR(200)	NOT NULL	Title of the portfolio item
description	TEXT	NULLABLE	Detailed description of the portfolio item
file_url	VARCHAR(500)	NULLABLE	URL to uploaded file (PDF or image)
external_url	VARCHAR(500)	NULLABLE	External link (e.g., GitHub, Behance)
created_at	TIMESTAMP	DEFAULT NOW()	Upload timestamp

4.2.4 Work History Table

Field Name	Data Type	Constraints	Description
entry_id	UUID	PK, NOT NULL	Unique identifier for the work history entry
user_id	UUID	FK (Users), NOT NULL	References the freelancer
project_name	VARCHAR(200)	NOT NULL	Name of the project or engagement
client_name	VARCHAR(200)	NULLABLE	Name of the client or organization
start_date	DATE	NOT NULL	Project start date
end_date	DATE	NULLABLE	Project end date (null if ongoing)
description	TEXT	NULLABLE	Summary of work performed

Field Name	Data Type	Constraints	Description
created_at	TIMESTAMP	DEFAULT NOW()	Entry creation timestamp

4.2.5 Reviews Table

Field Name	Data Type	Constraints	Description
review_id	UUID	PK, NOT NULL	Unique identifier for the review
freelancer_id	UUID	FK (Users), NOT NULL	References the freelancer being reviewed
client_id	UUID	FK (Users), NOT NULL	References the client submitting the review
rating	SMALLINT	CHECK (1-5), NOT NULL	Star rating between 1 and 5
review_text	TEXT	NULLABLE	Written feedback from the client
created_at	TIMESTAMP	DEFAULT NOW()	Review submission timestamp

5. Component Design

This section provides pseudocode-level descriptions of the key algorithms and logic for each major component of Module 1.

5.1 RegisterController

```
FUNCTION registerUser(request):
  INPUT: { email, password, role, full_name }
  VALIDATE: email format, password length >= 8, role in [freelancer, client, admin]
  IF validation fails THEN RETURN 400 Bad Request with error messages
  CHECK: email already exists in Users table
  IF email exists THEN RETURN 409 Conflict
  hash = bcrypt.hash(password, saltRounds=10)
  INSERT INTO Users (user_id, email, password_hash, role, is_active, created_at)
  INSERT INTO Profiles (profile_id, user_id, full_name)
  RETURN 201 Created with { user_id, email, role }
```

5.2 LoginController

```
FUNCTION loginUser(request):
  INPUT: { email, password }
  FIND user WHERE email = request.email AND is_active = true
  IF user not found THEN RETURN 401 Unauthorized
  IF bcrypt.compare(password, user.password_hash) == false THEN RETURN 401
  token = JWT.sign({ user_id, email, role }, SECRET_KEY, expiresIn='24h')
  RETURN 200 OK with { token, user_id, role }
```

5.3 AuthMiddleware

```
FUNCTION verifyToken(request, next):
  token = request.headers['Authorization'].split('Bearer ')[1]
  IF token is null THEN RETURN 401 Unauthorized
  decoded = JWT.verify(token, SECRET_KEY)
  IF verification fails THEN RETURN 403 Forbidden
  request.user = { user_id: decoded.user_id, role: decoded.role }
  CALL next()
```

5.4 ProfileController

```
FUNCTION getProfile(request):
  profile = SELECT * FROM Profiles WHERE user_id = request.params.userId
  IF not found THEN RETURN 404 Not Found
  RETURN 200 OK with profile data

FUNCTION updateProfile(request):
  REQUIRE: AuthMiddleware (validated JWT)
  IF request.user.user_id != request.params.userId THEN RETURN 403 Forbidden
  UPDATE Profiles SET (bio, location, skills, profile_picture_url, updated_at)
  RETURN 200 OK with updated profile
```

5.5 PortfolioController

```
FUNCTION addPortfolioItem(request):
  REQUIRE: AuthMiddleware, role = 'freelancer'
  VALIDATE: title is not empty, at least one of file or external_url provided
  IF file provided THEN
    VALIDATE: file type in [pdf, jpg, jpeg, png], size <= 10MB
    file_url = FileUploadHandler.save(file)
  INSERT INTO Portfolio_Items (item_id, user_id, title, description, file_url,
external_url)
  RETURN 201 Created with item data

FUNCTION deletePortfolioItem(request):
  REQUIRE: AuthMiddleware
  item = SELECT * FROM Portfolio_Items WHERE item_id = request.params.itemId
  IF item.user_id != request.user.user_id THEN RETURN 403 Forbidden
  DELETE FROM Portfolio_Items WHERE item_id = request.params.itemId
  RETURN 200 OK
```

5.6 ReviewController

```
FUNCTION submitReview(request):
  REQUIRE: AuthMiddleware, role = 'client'
  INPUT: { freelancer_id, rating, review_text }
  VALIDATE: rating is integer between 1 and 5
  VALIDATE: freelancer_id references a valid user with role = 'freelancer'
  INSERT INTO Reviews (review_id, freelancer_id, client_id, rating, review_text)
  new_avg = SELECT AVG(rating) FROM Reviews WHERE freelancer_id =
request.body.freelancer_id
  UPDATE Profiles SET avg_rating = new_avg WHERE user_id =
request.body.freelancer_id
  RETURN 201 Created
```

6. Human Interface Design

6.1 Overview of User Interface

The user interface for Module 1 is a web-based Single Page Application (SPA) built with React.js. The UI is designed to be clean, professional, and accessible across desktop and mobile browsers. It follows a consistent layout with a top navigation bar, a main content area, and contextual action buttons. The system provides six primary screens: Registration, Login, Profile Dashboard, Public Profile, Portfolio Management, and Reviews.

Navigation between screens is handled by React Router. Protected screens require a valid JWT token. Unauthenticated users attempting to access protected routes are redirected to the Login Page.

6.2 Screen Images

The following wireframes illustrate the interface from the user's perspective for each primary screen of Module 1. These wireframes represent the intended layout, key UI elements, and visual hierarchy of the module.

6.2.1 Screen 1 — Registration Page

The Registration Page is the entry point for new users. It presents a centered card with a role selector (Freelancer / Client), form fields for name, email, password and confirmation, a terms agreement checkbox, and a primary call-to-action button. Inline validation messages appear below each field on error.

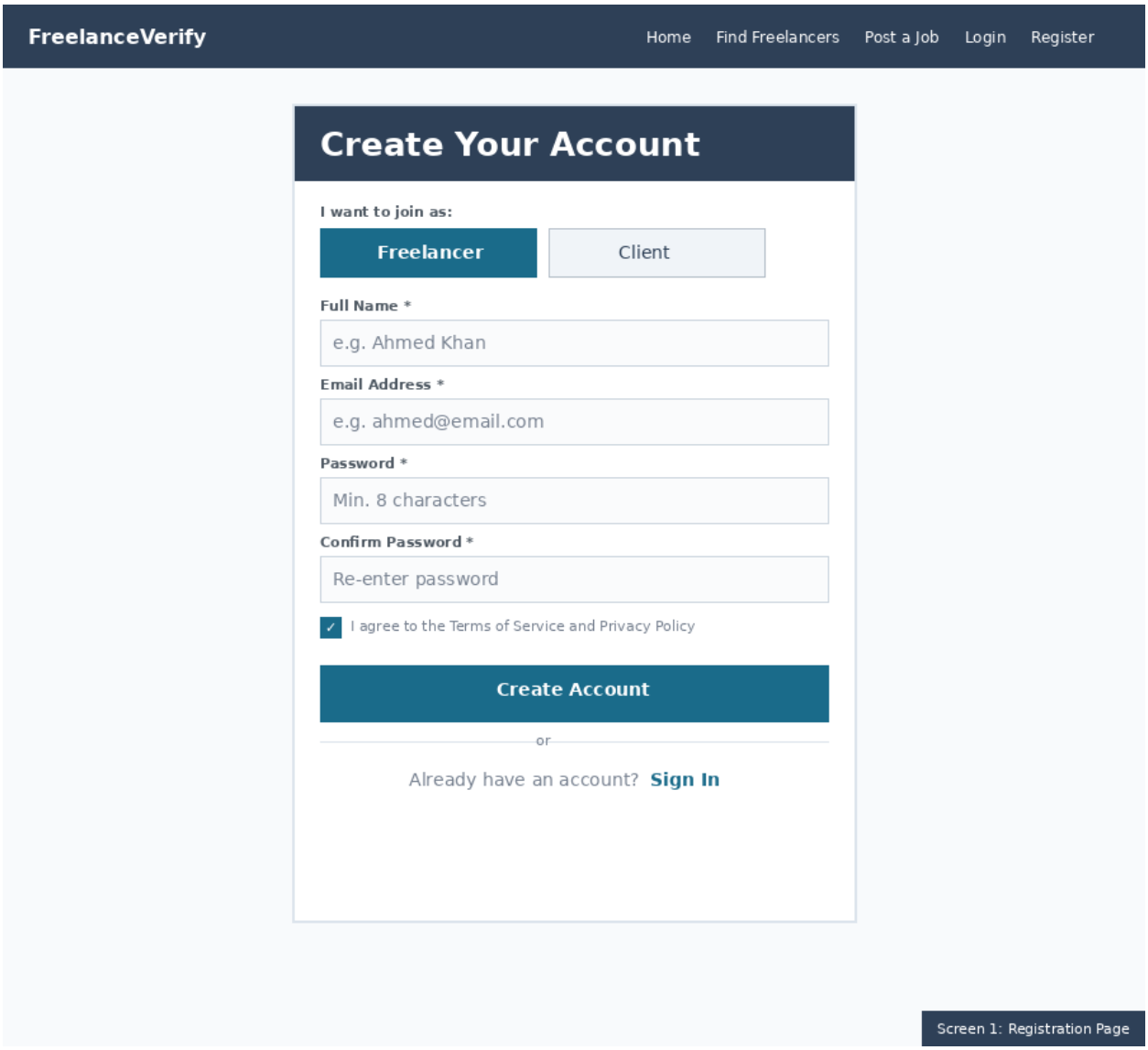

Figure 1: Registration Page — Multi-role account creation form

6.2.2 Screen 2 — Login Page

The Login Page provides a minimal, focused interface for returning users. It includes email and password fields, a forgot password link, a Sign In button, and a link to the Registration Page for new users.

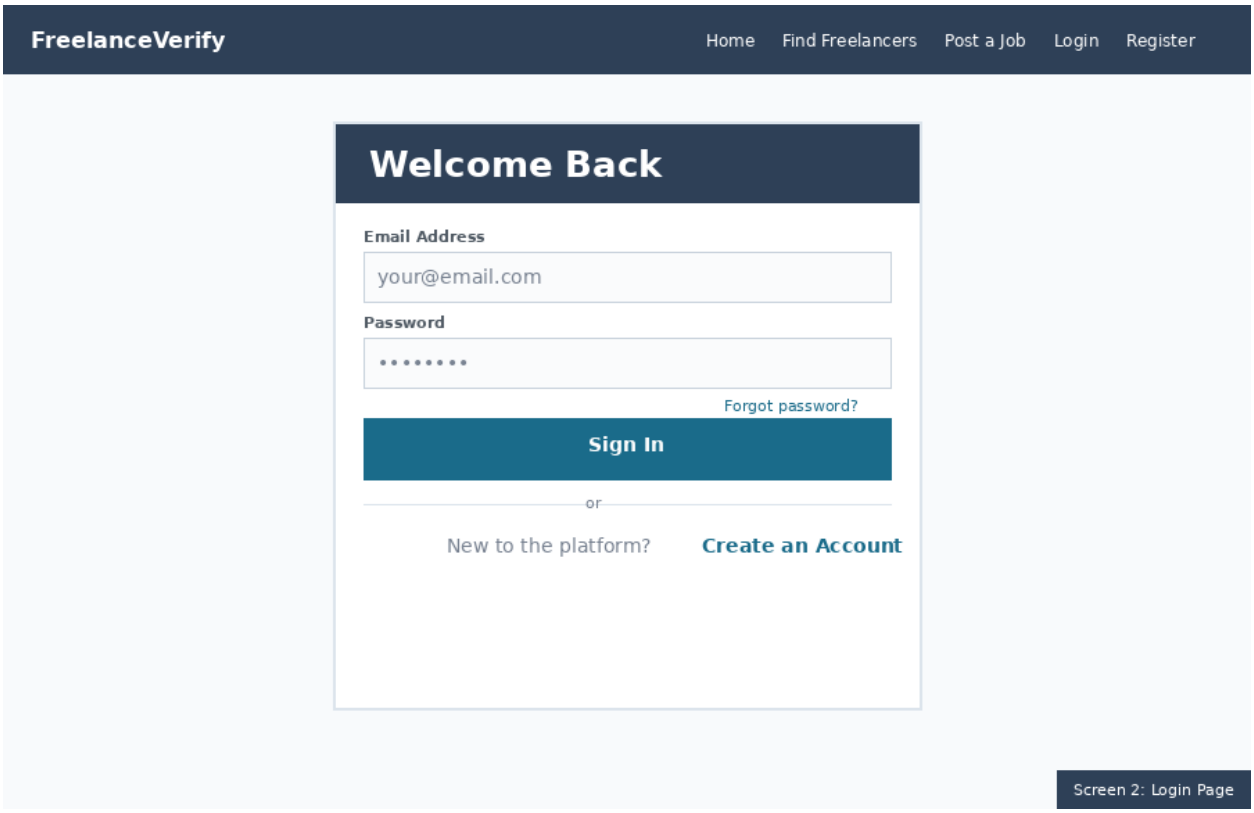

Figure 2: Login Page — Credential entry with error feedback

6.2.3 Screen 3 — Profile Dashboard (Logged-In View)

The Profile Dashboard is the authenticated user's personal management view. It features a left sidebar for navigation between module sections. The main area displays the user's profile picture with an upload control, editable name and location fields, a bio text area, a skills tag editor, and a received badges panel showing certifications from other platform modules.

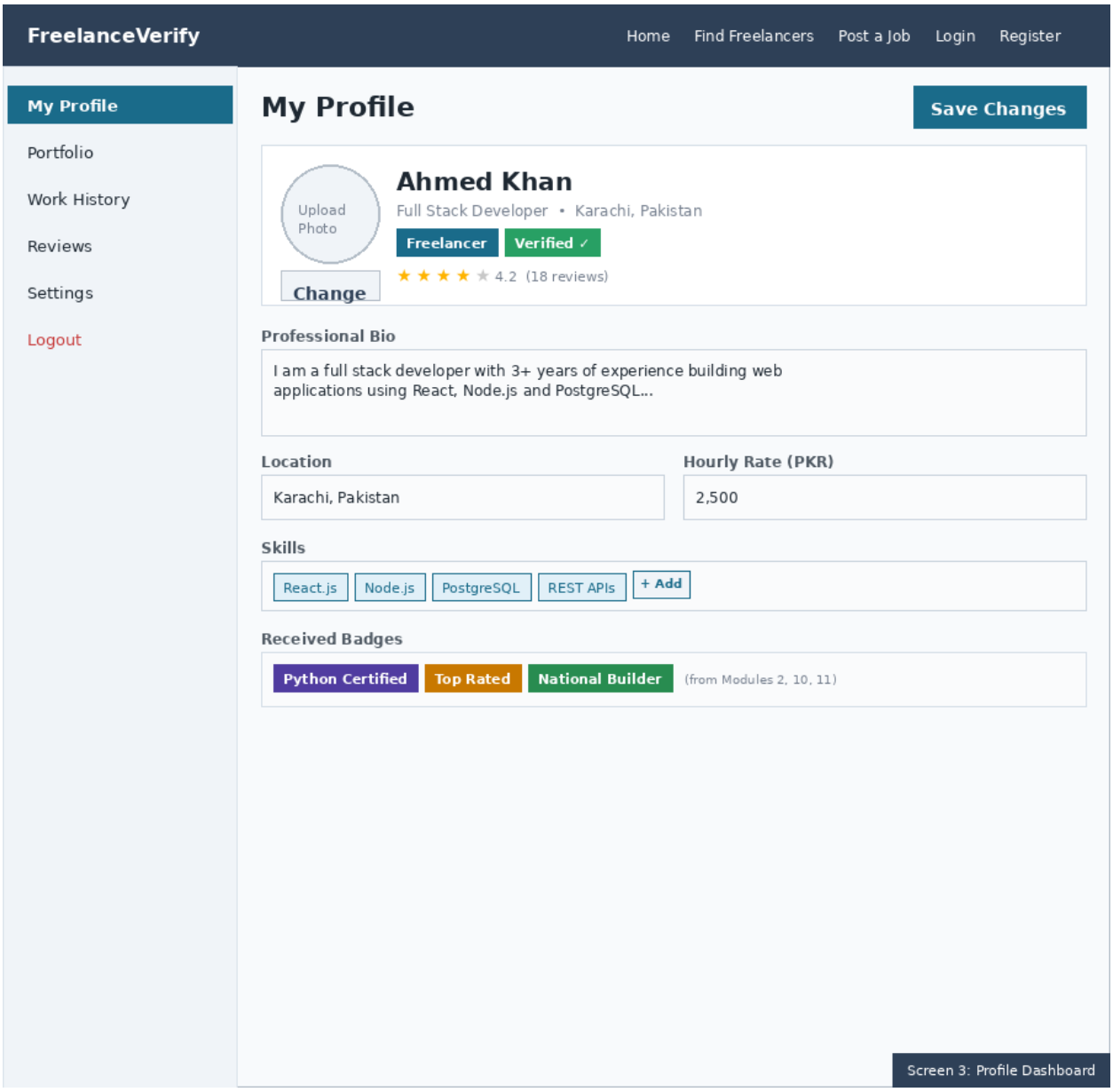

Figure 3: Profile Dashboard — Authenticated profile editing interface

6.2.4 Screen 4 — Public Profile Page

The Public Profile Page is the read-only view visible to clients, visitors, and other users. A hero banner displays the freelancer's avatar, name, location, and average star rating alongside a Hire button. Below the banner, the main column shows About, Skills, Badges, Portfolio cards, Work History entries, and Client Reviews. A right sidebar panel shows quick stats and a Leave a Review button for logged-in clients.

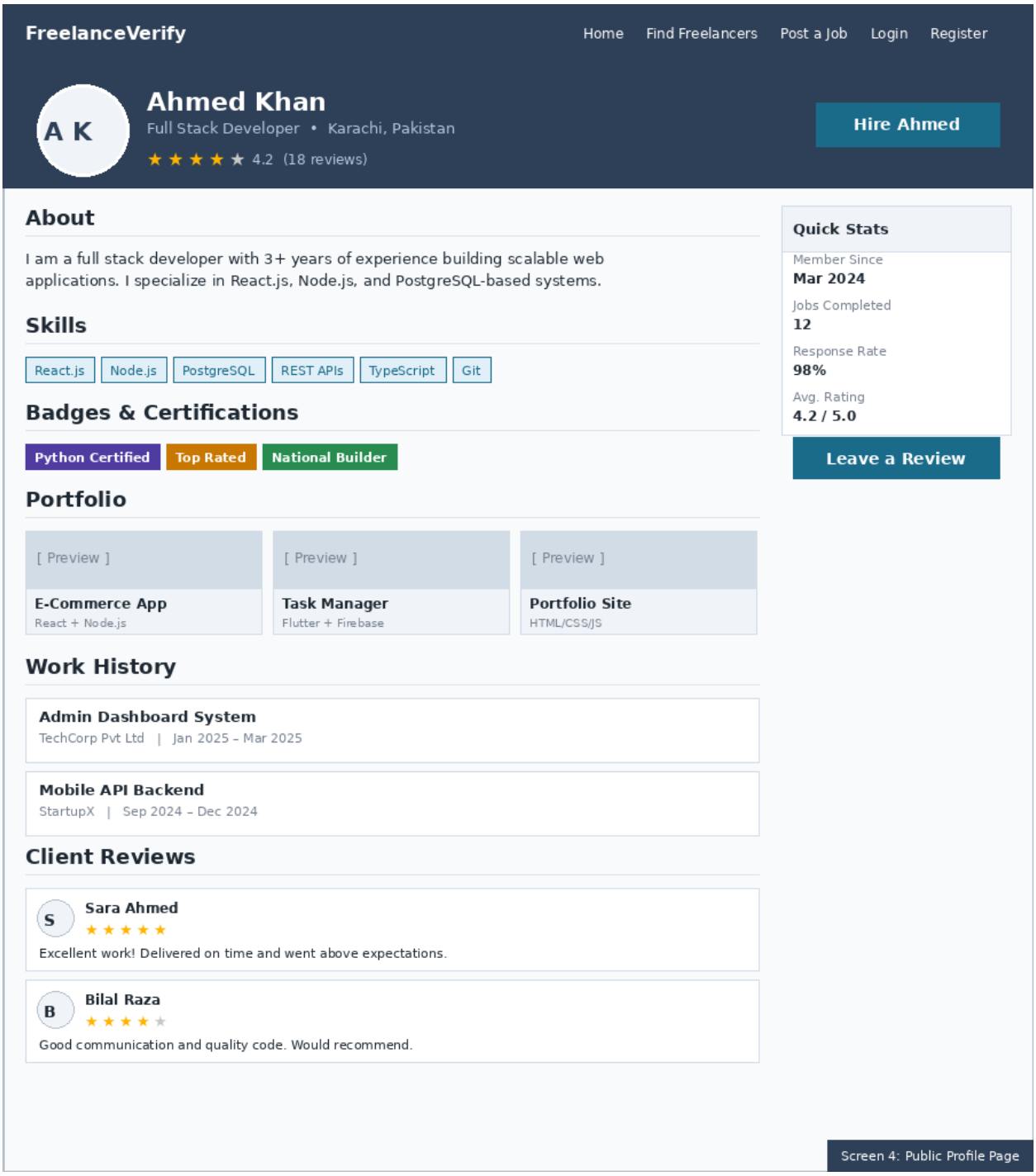

Figure 4: Public Profile Page — Freelancer's public-facing profile view

6.2.5 Screen 5 — Portfolio Management Panel

The Portfolio Management Panel allows freelancers to add, edit, and delete their portfolio items. An 'Add New Item' button opens an inline form for entering a title, uploading a file (PDF or image), or providing an external URL. Below the form, existing portfolio items are displayed as thumbnail cards with Edit and Delete action buttons.

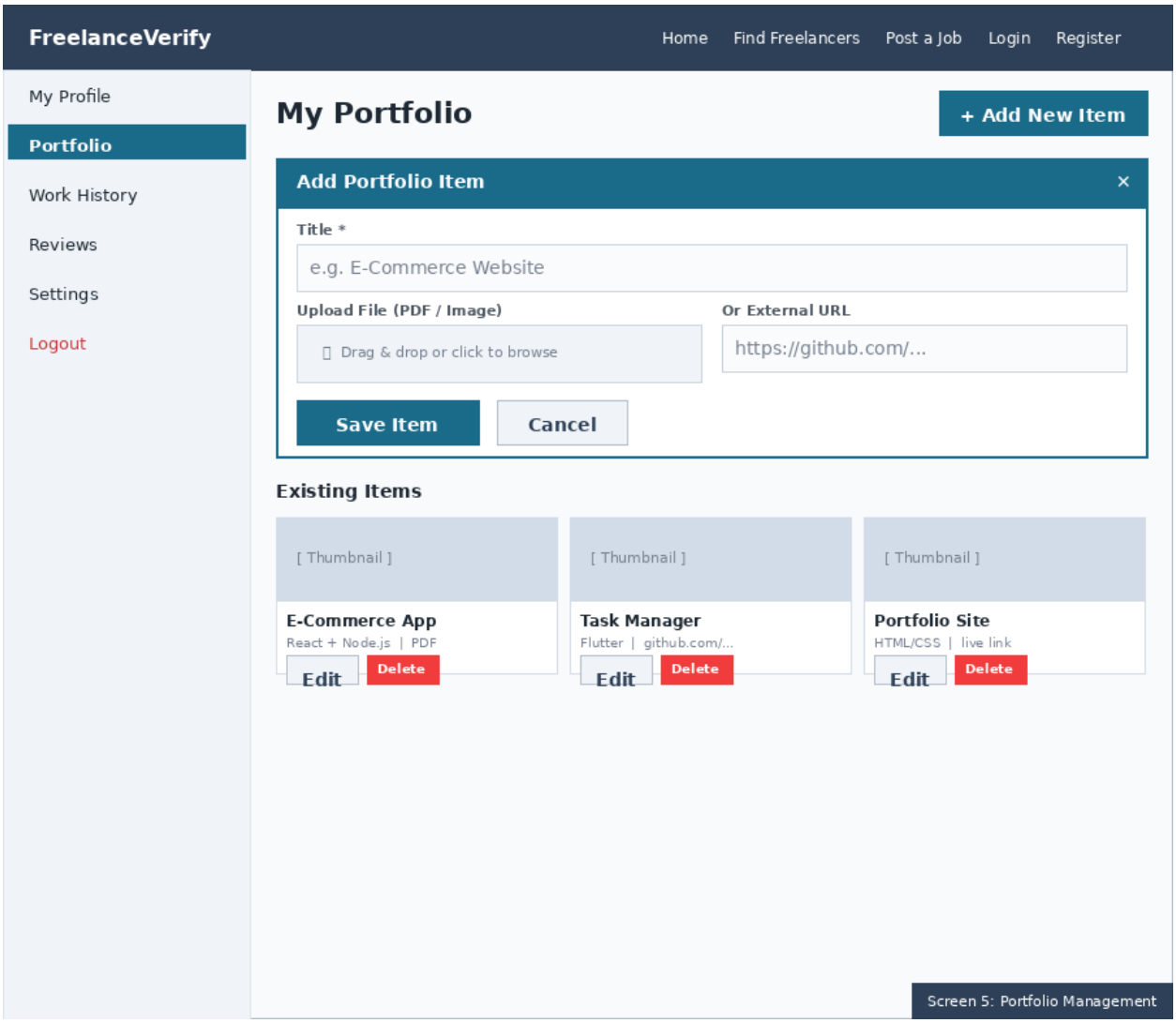

Figure 5: Portfolio Management Panel — Upload and manage portfolio items

6.2.6 Screen 6 — Reviews and Ratings

The Reviews and Ratings screen displays an aggregate rating summary with a star distribution bar chart, a review submission form (visible to logged-in clients only) with a clickable star selector and a text area, and a scrollable list of all submitted client reviews with reviewer avatars and star ratings.

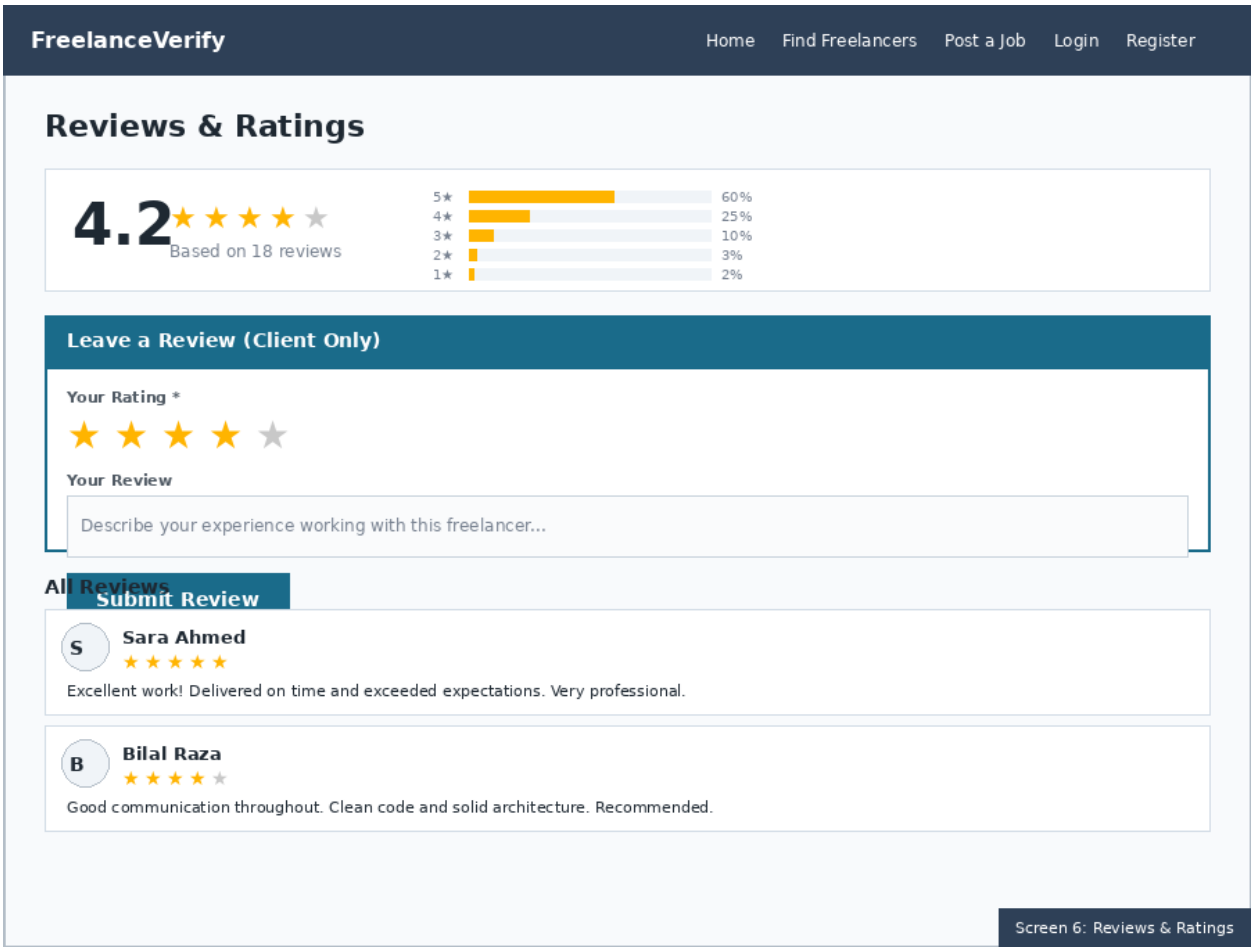

Figure 6: Reviews & Ratings — Aggregate summary, submission form, and review list

6.3 Screen Objects and Actions

Screen	UI Object	Action / Behaviour
Registration	Role Selector (toggle)	Switches between Freelancer and Client; sets the role field on form submission
Registration	Create Account Button	Submits form; triggers field-level validation; shows inline errors or redirects to dashboard on success
Login	Sign In Button	Authenticates credentials; stores JWT token in memory; redirects to profile dashboard
Login	Forgot Password Link	Navigates to password reset flow (TBD)
Profile Dashboard	Edit Profile / Save Changes	Toggles form between read and edit mode; saves updated profile to backend on confirm
Profile Dashboard	Profile Picture Upload	Opens OS file selector; validates image format and size; shows preview before saving
Profile Dashboard	Skills Tag Editor	Allows adding/removing skill tags inline; each tag is individually deletable

Screen	UI Object	Action / Behaviour
Portfolio Panel	Add New Item Button	Opens the add item form inline; collapses on cancel or successful save
Portfolio Panel	File Upload / Drag & Drop	Accepts PDF and image files up to 10MB; previews filename after selection
Portfolio Panel	Delete Button	Shows a confirmation dialog; on confirm, removes the item from display and database
Public Profile	Hire Button	Redirects the client to the job posting/messaging flow in Module 3 or 6
Public Profile	Leave a Review Button	Visible to logged-in clients only; opens the review submission form
Reviews Screen	Star Rating Selector	Interactive five-star click control; selected stars fill in yellow; updates rating value on submit
Reviews Screen	Submit Review Button	Validates that a rating is selected; posts review to backend; refreshes review list

7. Requirements Matrix

The following table traces each functional requirement from the SP-SRS to the design component(s) responsible for satisfying it.

Req ID	Requirement Description	Component(s)	Section Ref
FR-PM-01	User registration via email	RegisterController, Users Table	5.1, 4.2.1
FR-PM-02	Multi-role registration	RegisterController, Users.role	5.1, 4.2.1
FR-PM-03	Input validation on registration	RegisterController (validation logic)	5.1
FR-PM-04	Secure authentication	LoginController, bcrypt	5.2
FR-PM-05	JWT token issuance	LoginController, TokenService	5.2
FR-PM-06	User logout	AuthMiddleware (token invalidation)	5.3
FR-PM-07	Role-based access control	AuthMiddleware, RBACMiddleware	5.3, 3.2.6
FR-PM-08	Profile creation	RegisterController, Profiles Table	5.1, 4.2.2
FR-PM-09	Profile editing	ProfileController.updateProfile	5.4
FR-PM-10	Public profile display	ProfileController.getProfile	5.4, 6.2.4
FR-PM-11	Profile picture upload	ProfileController, FileUploadHandler	5.4, 6.2.3
FR-PM-12	Profile edit authorization	ProfileController (ownership check)	5.4
FR-PM-13	Badges display on profile	Profiles.badges (JSONB), Public Profile UI	4.2.2, 6.2.4
FR-PM-14	Portfolio item upload	PortfolioController.addPortfolioItem	5.5, 4.2.3
FR-PM-15	Portfolio title & description	Portfolio_Items Table	4.2.3
FR-PM-16	Portfolio edit/delete	PortfolioController.deletePortfolioItem	5.5
FR-PM-17	Portfolio public display	Public Profile UI (Portfolio Section)	6.2.4
FR-PM-18	Portfolio owner authorization	PortfolioController (ownership check)	5.5
FR-PM-19	Add work history entry	WorkHistoryController, Work_History Table	3.2.4, 4.2.4
FR-PM-20	Edit/delete work history	WorkHistoryController (CRUD)	3.2.4
FR-PM-21	Work history display	Public Profile UI (Work History Section)	6.2.4
FR-PM-22	Star rating submission	ReviewController.submitReview	5.6
FR-PM-23	Written review submission	ReviewController, Reviews Table	5.6, 4.2.5
FR-PM-24	Average rating calculation	ReviewController, Profiles.avg_rating	5.6, 4.2.2
FR-PM-25	Reviews display on profile	Public Profile UI (Reviews Section)	6.2.4, 6.3

Req ID	Requirement Description	Component(s)	Section Ref
FR-PM-26	Restrict reviews to clients	ReviewController (RBAC role check)	5.6, 5.3

8. Appendices

Appendix A: API Endpoint Summary

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register a new user (freelancer/client)
POST	/api/auth/login	No	Login and receive JWT token
POST	/api/auth/logout	Yes (JWT)	Logout and invalidate session
GET	/api/profiles/:userId	No	Get public profile of any user
PUT	/api/profiles/:userId	Yes (JWT, Owner)	Update own profile
POST	/api/profiles/:userId/picture	Yes (JWT, Owner)	Upload profile picture
GET	/api/portfolio/:userId	No	Get all portfolio items for a user
POST	/api/portfolio	Yes (JWT, Freelancer)	Add a new portfolio item
PUT	/api/portfolio/:itemId	Yes (JWT, Owner)	Edit a portfolio item
DELETE	/api/portfolio/:itemId	Yes (JWT, Owner)	Delete a portfolio item
GET	/api/workhistory/:userId	No	Get work history for a user
POST	/api/workhistory	Yes (JWT, Freelancer)	Add a work history entry
PUT	/api/workhistory/:entryId	Yes (JWT, Owner)	Edit a work history entry
DELETE	/api/workhistory/:entryId	Yes (JWT, Owner)	Delete a work history entry
GET	/api/reviews/:userId	No	Get all reviews for a freelancer
POST	/api/reviews	Yes (JWT, Client)	Submit a review for a freelancer
PUT	/api/profiles/:userId/badges	Yes (Module Token)	Update badges from external modules

Appendix B: Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js 18+	Single Page Application UI
Frontend	React Router	Client-side navigation and protected routes
Backend	Node.js 18+	Server runtime environment

Layer	Technology	Purpose
Backend	Express.js	RESTful API framework
Database	PostgreSQL	Shared centralized database; Module 1 owns the Users, Profiles, Portfolio Items, Work History, and Reviews tables within the shared schema
Dev Tools	GitHub	Version control and collaboration
Communication	HTTPS / REST / JSON	All API communication protocol